

2. Connect Real AI/ML Models (Edge + Cloud)

Target Goal: Replace simulated AI data with real deep learning models: an offline edge classifier using ONNX Web for zero-connectivity fields, and a cloud-based FastAPI YOLOv8 service for bounding-box disease localization.

Step 1: Dataset & Model Strategy

Model Type	Architecture	Dataset Sources	Latency & Size
Disease Classifier (Edge)	MobileNetV3-Large / EfficientNet-B0	PlantVillage (54k images) + ICAR field samples	<150ms on mobile CPU (~12 MB INT8)
Lesion & Pest Detector (Cloud)	YOLOv8-nano (<code>`yolov8n.pt`</code>)	Roboflow Agricultural Disease Dataset (annotated boxes)	~80ms on Cloud GPU / ~300ms CPU

Step 2: Production FastAPI Service (`backend/main.py`)

Create your Python backend using FastAPI and Ultralytics YOLO:

```

# backend/main.py
from fastapi import FastAPI, UploadFile, File, Form, HTTPException
from fastapi.middleware.cors import CORSMiddleware
from PIL import Image
import io
import imagehash
from ultralytics import YOLO

app = FastAPI(title="CropCare AI Inference Engine", version="2.0")

# Enable CORS for frontend domain
app.add_middleware(
    CORSMiddleware,
    allow_origins=["*"],
    allow_methods=["*"],
    allow_headers=["*"],
)

# Load pre-trained / fine-tuned YOLO model
model = YOLO("yolov8n-crop-diseases.pt")

# In-memory duplicate hash store (replace with Redis/Postgres in Step 4)
SEEN_HASHES = set()

@app.post("/api/v1/scan")
async def scan_crop(file: UploadFile = File(...), crop: str = Form("Tomato")):
    if not file.content_type.startswith("image/"):
        raise HTTPException(status_code=400, detail="Uploaded file is not an image")

    contents = await file.read()
    image = Image.open(io.BytesIO(contents)).convert("RGB")

    # 1. Duplicate Image Detection via Perceptual Hash
    img_hash = str(imagehash.dhash(image))
    is_duplicate = img_hash in SEEN_HASHES
    SEEN_HASHES.add(img_hash)

    # 2. Run YOLO Object Detection for Disease Lesions
    results = model(image)
    detections = []
    total_area_pixels = image.width * image.height
    affected_pixels = 0

    for r in results:
        for box in r.boxes:
            x1, y1, x2, y2 = box.xyxy[0].tolist()
            conf = float(box.conf[0])
            cls_id = int(box.cls[0])
            name = model.names[cls_id]

            box_area = (x2 - x1) * (y2 - y1)
            affected_pixels += box_area

            detections.append({
                "disease": name,
                "confidence": round(conf * 100, 1),
                "bbox": [round(x1), round(y1), round(x2), round(y2)]
            })

    affected_percentage = min(100.0, round((affected_pixels / total_area_pixels) * 100, 1))

    return {

```

```

    "status": "success",
    "is_duplicate": is_duplicate,
    "image_hash": img_hash,
    "crop": crop,
    "detections_count": len(detections),
    "detections": detections,
    "severity_score": min(100, int(affected_percentage * 1.4)),
    "affected_area_percentage": affected_percentage
  }

```

Step 3: Exporting Model to ONNX for 100% Offline Browser Inference

To run the model directly inside a farmer's Chrome browser with zero internet, export and quantize your model to ONNX format:

```

# Run in Python to create lightweight web model
import torch
import torchvision.models as models

# Example: Export MobileNetV3
model = models.mobilenet_v3_small(pretrained=True)
dummy_input = torch.randn(1, 3, 224, 224)

torch.onnx.export(
    model, dummy_input, "cropcare_mobilenet.onnx",
    input_names=['input'], output_names=['output'],
    opset_version=13
)
print("Exported cropcare_mobilenet.onnx")

```

Step 4: Frontend In-Browser Inference via `onnxruntime-web`

Install `onnxruntime-web` in your frontend:

```
npm install onnxruntime-web
```

```

// src/services/edgeInference.ts
import * as ort from 'onnxruntime-web';

export async function runLocalLeafDiagnosis(imageElement: HTMLImageElement) {
  // Load model cached by PWA Service Worker
  const session = await ort.InferenceSession.create('/models/cropcare_mobilenet.onnx', {
    executionProviders: ['wasm']
  });

  // Preprocess image tensor to 224x224 normalized RGB
  const tensor = preprocessImageToTensor(imageElement);
  const feeds = { input: tensor };
  const results = await session.run(feeds);
  const output = results.output.data;

  return parseProbabilities(output);
}

```

